

| 853. 有边数限制的最短路

|  [853 - 有边数限制的最短路](#)

| 题目描述

| (请在此处粘贴题目描述)

给定一个 n 个点 m 条边的有向图，图中可能存在重边和自环，**边权可能为负数**。

请你求出从 1 号点到 n 号点的最多经过 k 条边的最短距离，如果无法从 1 号点走到 n 号点，输出 **impossible**。

注意：图中可能 **存在负权回路**。

| 输入格式

第一行包含三个整数 n, m, k 。

接下来 m 行，每行包含三个整数 x, y, z ，表示存在一条从点 x 到点 y 的有向边，边长为 z 。

点的编号为 $1 \sim n$ 。

| 输出格式

输出一个整数，表示从 1 号点到 n 号点的最多经过 k 条边的最短距离。

如果不存在满足条件的路径，则输出 **impossible**。

| 数据范围

$1 \leq n, k \leq 500$,

$1 \leq m \leq 10000$,

$1 \leq x, y \leq n$,

任意边长的绝对值不超过 10000。

输入样例：

```
3 3 1
1 2 1
2 3 1
1 3 3
```

输出样例：

```
3
```

💡 解题思路

Bellman-ford 算法：

for n 次：迭代 k 次表示最多不经过 k 条边，即从 1 号点经过不超过 k 条边走到所有点的最短距离

for 所有边 a, b, w

$\text{dist}[b] = \min(\text{dist}[b], \text{dist}[a] + w)$

 （可能会涉及到 **backup** 数组，存放上一轮的 **dist** 数组，防止串联效应）

- 算法证明了：循环结束后 $\text{dist}[b] \leq \text{dist}[a] + w$
- 该算法可以处理有负权边的图，但是当负权回路上出现在路径上时，不存在最短路
- 如果当第 n 次迭代还有边更新的话，则说明存在一条边数为 n 的最短路径，则一定存在负环

// 求1到n的最短路距离，如果无法从1走到n，则输出"impossible"。

```
int bellman_ford()
```

```
{
```

```
    memset(dist, 0x3f, sizeof dist);
```

```
    dist[1] = 0;
```

// 如果第n次迭代仍然会松弛三角不等式，就说明存在一条长度是n的最短路径，则会经过n+1个点，由抽屉原理，路径中至少存在两个相同的点，说明图中存在负权回路。

```
for (int i = 0; i < n; i ++ ) // 迭代n次即可
{
    for (int j = 0; j < m; j ++ )
    {
        int a = edges[j].a, b = edges[j].b, w = edges[j].w;
        if (dist[b] > dist[a] + w)
            dist[b] = dist[a] + w;
    }
}

//可能会有负权边 但是不会减少太多 最多减500*10000
if(dist[n] > 0x3f3f3f3f / 2) puts("impossible");
else cout << dist[n] << endl;
}
```

代码实现

my answer

```
#include <iostream>
#include <cstring>

using namespace std;
const int N = 510, M = 10010;
int n, m, k;
int dist[N], backup[N]; // dist:距离数组 backup:上一次dist数组的值
struct Edge {
    int a, b, w;
} edges[M]; // 存储边

void bellman_ford()
{
    memset(dist, 0x3f, sizeof(dist)); // 初始化距离数组
    dist[1] = 0; // 起点距离为0

    // k次循环
```

```

    for (int i = 0; i < k; i++) {
        memcpy(backup, dist, sizeof(dist)); // 拷贝上一次的dist数组
        // 遍历所有边
        for (int j = 0; j < m; j++) {
            int a = edges[j].a, b = edges[j].b, w = edges[j].w;
            // 若不使用backup更新 可能是刚被更新的距离又去更新其他距离
            // 这样一次迭代就造成了超过1次的更新 即可能更新多条边
            dist[b] = min(dist[b], backup[a] + w);
        }
    }
    // 可能会有负权边 但是不会减少太多 最多减500*10000
    if (dist[n] > 0x3f3f3f3f / 2) { // 负权边会将dist[n]更新 所以不能
    使用 dist[n] == 0x3f3f3f3f 来判定
        puts("impossible");
    } else {
        cout << dist[n] << endl;
    }
}

int main()
{
    scanf("%d%d%d", &n, &m, &k);

    for (int i = 0; i < m; i++) {
        int a, b, w;
        scanf("%d%d%d", &a, &b, &w);
        edges[i] = {a, b, w};
    }

    bellman_ford();

    return 0;
}

```

- **时间复杂度：** $O(k * m)$
- **空间复杂度：** $O(m)$

to optimize

- **时间复杂度：** $O(\dots)$
- **空间复杂度：** $O(\dots)$

🌟 tips

others' answer

- **时间复杂度：** $O(\dots)$
- **空间复杂度：** $O(\dots)$

🌟 tips

复盘与扩展

关键点：

易错点：

相关题目

🕒 本次耗时： ____ 分钟